

EECE527

INTRODUCTION TO HAPTICS

LECTURE 15: COLLISION DETECTION (6)

2011-04-28
Seungmoon Choi, Ph.D.

Basic Primitive Test

- ▶ After a high-level system has ruled out as many objects as possible from further collision tests, all collision systems must perform low-level tests between primitives or bounding volumes to determine intersection status.
- ▶ Chapter 5 in your text describes how these low-level tests can be efficiently performed.
- ▶ Topics
 - ▶ Closest-point computations
 - ▶ Testing whether primitives are intersecting
 - ▶ Intersecting lines, rays, and (directed) segments
 - ▶ Additional tests
 - ▶ Dynamic intersection tests
- ▶ Study by yourself!

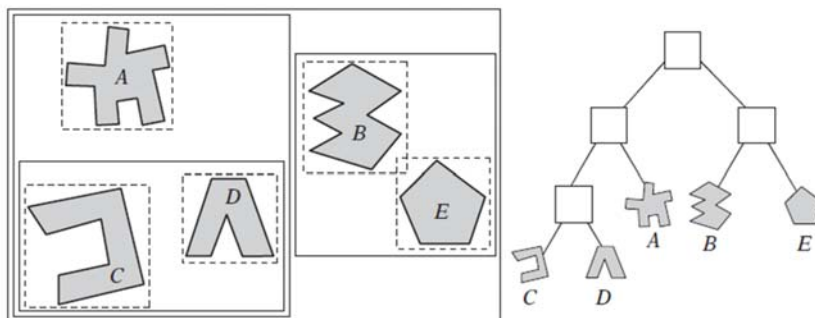
BOUNDING VOLUME HIERACHY (1)

▶ 3

Bounding Volume Hierarchies - Introduction

▶ Motivations

- ▶ Using bounding volumes can result in significant performance improvements.
- ▶ However, the same number of pairwise tests still has to be performed, so the asymptotic time complexity remains the same.
- ▶ By arranging the bounding volumes into a tree hierarchy called a bounding volume hierarchy (BVH), the time complexity can be reduced to logarithmic in the number of tests to perform.



▶ 4

Hierarchy Design Issues (1)

- ▶ Desired BVH Characteristics
 - ▶ The nodes contained in any given subtree should be near each other.
 - ▶ Each node in the hierarchy should be of minimal volume.
 - ▶ The sum of all bounding volumes should be minimal.
 - ▶ Greater attention should be paid to nodes near the root of the hierarchy.
 - ▶ Pruning a node near the root of the tree removes more objects from further consideration than removal of a deeper node would.
 - ▶ The volume of overlap of sibling nodes should be minimal.
 - ▶ The hierarchy should be balanced with respect to both its node structure and its content.
 - ▶ Balancing allows as much of the hierarchy as possible to be pruned whenever a branch is not traversed into.
- ▶ Other Desired BVH Characteristics
 - ▶ Real-time applications: Guaranteeing the worst-case time
 - ▶ Total memory requirements

▶ 5

Hierarchy Design Issues (2)

- ▶ Cost Functions
 - ▶ T : Total cost of intersecting the two hierarchies
 - ▶ N_V : The number of BV pairs tested for overlap
 - ▶ C_V : The cost of testing a pair of BVs for overlap
 - ▶ N_P : The number of primitive pairs tested
 - ▶ C_P : The cost of testing a primitive pair
 - ▶ N_U : The number of nodes that need to be updated
 - ▶ C_U : The cost of updating each such node
 - ▶ C_O : The cost of a one-time processing (such as a coordinate transformation between objects).

$$T = N_V C_V + N_P C_P + N_U C_U + C_O.$$

- ▶ In general, the values are so intrinsically linked that minimizing one value often causes another to increase.

▶ 6

Hierarchy Design Issues (3)

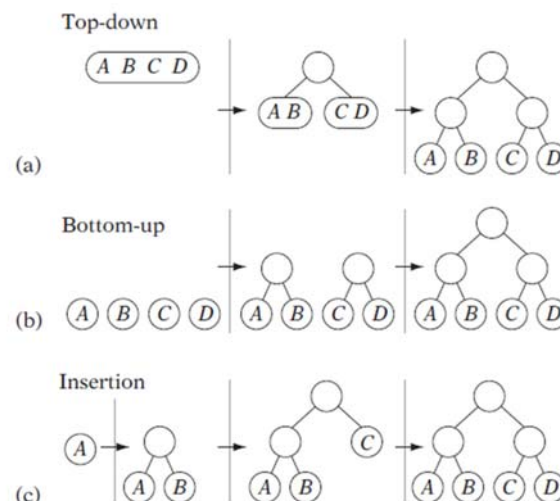
▶ Tree Degree

- ▶ What degree or branching factor should be used in the tree representing the bounding volume hierarchy?
- ▶ This question of what degree to use is a difficult one and no definitive answer has been forthcoming.
- ▶ Looking at actual usage, it appears binary trees are by far the most common hierarchical representation.
- ▶ Binary trees are easier to build and, to some extent, to represent and traverse than other trees.
- ▶ There also exist some theoretical support for the optimality of binary tree.

▶ 7

Building Strategies for Hierarchy Construction

- ▶ As the number of possible trees grows exponentially in terms of the number of elements in the input set, an exhaustive search for the best tree is infeasible.
- ▶ So we need some heuristic rules to guide tree construction.



▶ 8

Top-down Construction

```
// Construct a top-down tree. Rearranges object[] array during construction
void TopDownBVTree(Node **tree, Object object[], int numObjects)
{
    assert(numObjects > 0);

    const int MIN_OBJECTS_PER_LEAF = 1;
    Node *pNode = new Node;
    *tree = pNode;
    // Compute a bounding volume for object[0], ..., object[numObjects - 1]
    pNode->BV = ComputeBoundingVolume(&object[0], numObjects);
    if (numObjects <= MIN_OBJECTS_PER_LEAF) {
        pNode->type = LEAF;
        pNode->numObjects = numObjects;
        pNode->object = &object[0]; // Pointer to first object in leaf
    } else {
        pNode->type = NODE;
        // Based on some partitioning strategy, arrange objects into
        // two partitions: object[0..k-1], and object[k..numObjects-1]
        int k = PartitionObjects(&object[0], numObjects);
        // Recursively construct left and right subtree from subarrays and
        // point the left and right fields of the current node at the subtrees
        TopDownBVTree(&(pNode->left), &object[0], k);
        TopDownBVTree(&(pNode->right), &object[k], numObjects - k);
    }
}
```

9

Partitioning Strategies

- ▶ We need to decide how the input set is partitioned into two subsets.
- ▶ To simplify the partitioning, the set is usually divided into subsets using a splitting hyperplane.
- ▶ If a primitive is cut across by the splitting plane, we can use the position of its centroid with respect to the splitting plane to determine which subset it goes in.
- ▶ For partitioning, we can find one partitioning axis and one partitioning point.
- ▶ A plane at that point orthogonal to the axis is a splitting plane.

10

Choice of Partitioning Axis

- ▶ Since an infinite number of possible axes exist, the search is limited to a small number of axes from which the best one is picked.
- ▶ Common choices of axes
 - ▶ Local x, y, and z coordinate axes.
 - ▶ Axes from the intended aligned bounding volume.
 - ▶ Axes of the parent bounding volume.
 - ▶ Axis through the two most distant points.
 - ▶ Axis along which variance is greatest.
- ▶ Once a partitioning axis has been selected a small number of hill-climbing steps can be performed to improve on the axis.
 - ▶ One approach involves perturbing the direction of the axis, replacing the selected axis if the perturbed axis performs better, and repeating this for as many steps as can be afforded.

▶ 11

Choice of Split Point

- ▶ Again there exist infinitely many candidates for the split point. So we need to limit the number of points to test.
- ▶ Good candidates to test
 - ▶ Median of the centroid coordinates (object median).
 - ▶ Mean of the centroid coordinates (object mean).
 - ▶ Median of the bounding volume projection extents (spatial median).
 - ▶ Splitting at k evenly spaced points along the bounding volume projection extents.
 - ▶ Splitting between (random subset of) the centroid coordinates.

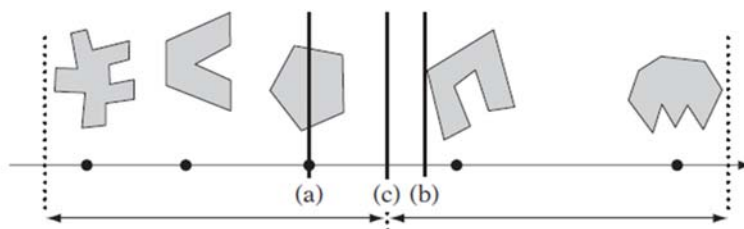


Figure 6.3 (a) Splitting at the object median. (b) Splitting at the object mean. (c) Splitting at the spatial median.

▶ 12

Bottom-up Construction (1)

```
Node *BottomUpBVTree(Object object[], int numObjects)
{
    assert(numObjects != 0);

    int i, j;

    // Allocate temporary memory for holding node pointers to
    // the current set of active nodes (initially the leaves)
    NodePtr *pNodes = new NodePtr[numObjects];

    // Form the leaf nodes for the given input objects
    for (i = 0; i < numObjects; i++) {
        pNodes[i] = new Node;
        pNodes[i]->type = LEAF;
        pNodes[i]->object = &object[i];
    }
    // Merge pairs together until just the root object left
    while (numObjects > 1) {
        // Find indices of the two "nearest" nodes, based on some criterion
        FindNodesToMerge(&pNodes[0], numObjects, &i, &j);
        // Group nodes i and j together under a new internal node
        Node *pPair = new Node;
```

► 13

Bottom-up Construction (2)

```
    pPair->type = NODE;
    pPair->left = pNodes[i];
    pPair->right = pNodes[j];
    // Compute a bounding volume for the two nodes
    pPair->BV = ComputeBoundingVolume(pNodes[i]->object, pNodes[j]->object);

    // Remove the two nodes from the active set and add in the new node.
    // Done by putting new node at index 'min' and copying last entry to 'max'
    int min = i, max = j;
    if (i > j) min = j, max = i;
    pNodes[min] = pPair;
    pNodes[max] = pNodes[numObjects - 1];
    numObjects--;
}
// Free temporary storage and return root of tree
Node *pRoot = pNodes[0];
delete pNodes;
return pRoot;
}
```

The complexity of a brute-force approach is $O(n^3)$.

► 14

Improved Bottom-up Construction (1)

```
Node *BottomUpBVTree(Object object[], int numObjects)
{
    PriorityQueue<Pair> q;
    InsertionBVTree t;

    // Bound all objects in BV, forming leaf nodes. Insert leaf nodes into a
    // dynamically changable insertion-built BV tree
    InitializeInsertionBVTree(t, object, numObjects);

    // For all nodes, form pair of references to the node and the node it pairs
    // best with (resulting in the smallest bounding volume). Add all pairs to
    // the priority queue, sorted on increasing volume order
    InitializePriorityQueue(q, object, numObjects);

    while (SizeOf(q) > 1) {
        // Fetch the smallest volume pair from the queue
        Pair *p = Dequeue(q);

        // Discard pair if the node has already been paired
        if (HasAlreadyBeenPaired(p->node)) continue;
    }
}
```

► 15

Improved Bottom-up Construction (2)

```
// Recompute the best pairing node for this node to see if
// the stored pair node is still valid
Node *bestpairnode = ComputeBestPairingNodeUsingTree(t, p->node);
if (p->pairnode == bestpairnode) {
    // The store pair node is OK, pair the two nodes together;
    // link the nodes together under a new node
    Node *n = new Node;
    n->left = p->node;
    n->right = p->pairnode;

    // Add new node to BV tree; delete old nodes as not possible to pair with
    Delete(t, p->node);
    Delete(t, p->pairnode);
    Insert(t, n);

    // Compute a pairing node for the new node; insert it into queue
    Node *newbestpairnode = ComputeBestPairingNodeUsingTree(t, n);
    p = Pair(n, newbestpairnode);
}
```

► 16

Improved Bottom-up Construction (3)

```
    } else {  
        // Best pair node changed since the pair was inserted;  
        // update the pair, reinsert into queue and try again  
        p = Pair(p->node, bestpairnode);  
    }  
    Enqueue(q, p, VolumeOfBVForPairedNodes(p));    // Queue, pair, priority  
}  
return Dequeue(q)->node;  
}
```

Also see the text for more bottom-up construction strategies.

▶ 17

Incremental (Insertion) Construction

- ▶ The tree is built by inserting one object at a time, starting from an empty tree.
- ▶ Objects are inserted by finding the insertion location in the tree that causes the tree to grow as little as possible according to a cost metric.
- ▶ Cost = An object's bounding volume + increase in volume its insertion causes in all ancestor volumes above it in the tree.
- ▶ Insertion order objects: Random shuffle before insertions
- ▶ Insertion strategies
 - ▶ Single root-leaf traversal: $O(n \log n)$
 - ▶ Examine the entire tree for each new object
- ▶ Relatively less work done than the top-down and bottom-up constructions

▶ 18

Comparisons of Construction Approaches

- ▶ Top-down construction
 - ▶ The most common approach
 - ▶ Easy implementation and fast tree construction.
 - ▶ Results are not as good as those of the others as critical decisions are made early in the algorithm at a point where all information is not available.
- ▶ Bottom-up construction
 - ▶ More complicated to implement and a slower construction time.
 - ▶ Usually produces the best trees
- ▶ Insertion construction
 - ▶ As fast as or even faster than top-down methods
 - ▶ Can produce better results than top-down methods
 - ▶ On-line: Good for dynamic systems

Hierarchy Traversal

- ▶ To determine the overlap status of two bounding volume hierarchies, some rule must be established for how to descend the trees when their top-level volumes overlap.
- ▶ Common tree traversal methods



Figure 6.4 (a) Breadth-first search, (b) depth-first search, and (c) one possible best-first search ordering.

- ▶ DFS seems to be the most popular choice for collision detection systems.
- ▶ DFS is often enhanced by a simple heuristic to guide the search along, improving on the basic blind DFS approach without the overhead of, say, a full best-first search.